ARCHITECTURAL WHITEPAPER

How Puck Visual Website Builder Works & Saves Data

A Simple, Easy-to-Understand Guide on the Data Storage, Styling Pipeline, and Neon Database Architecture

★ The 10-Second Analogy: Blueprint vs. Finished House

Imagine you are building a custom Lego house or ordering a tailored suit:

> **Instead of saving heavy, rigid, locked-down HTML and CSS files**, Puck saves a lightweight **digital recipe** (called JSON). It records only your exact choices: *"Add a Hero section; make background dark navy (#090d16); make corner radius 16px; fade up in 700ms."*

When someone visits your website, your Next.js application reads that recipe in milliseconds and assembles the live, interactive, animated website instantly on screen.

1 What Happens When You Customize in the Editor?

Inside the visual builder canvas and right sidebar:

- **Color Pickers:** When you pick `#6366f1` from the palette swatch, Puck updates the component prop `accentColor: "#6366f1"`.
- **Sliders & Radii:** Dragging a slider from 0 to 16 sets `borderRadius: 16`.
- **Multiple Products / Grids:** Choosing 4 columns with row scroll sets `layout: "row"` and `columns: "4"`.
- **Animations:** Selecting Fade-Up with 150ms delay sets `animPreset: "fade-up"` and `animDelay: 150`.

Every single adjustment lives inside a live JavaScript state object representing the page.

2 Where & How It Is Saved in Your Neon Database

When you click **"Save Template"** or **"Publish"**, your frontend sends the state to your backend API, which saves it into PostgreSQL (NeonDB):

Database Table	Column Name	Data Type	What It Actually Holds
Template	`id, name, slug`	**String / CUID**	Template metadata (e.g. "Agency Studio", "E-commerce")
TemplatePage	`editorData`	**JSONB**	**The Core Document:** Holds all components, props, text, and styles.
WebsitePage	`editorData`	**JSONB**	When a user clicks "Use Template", this copies the recipe to their website.

Here is an exact real-world sample of what is stored inside the `editorData` column in your Neon database:


```
{
  "content": [
    {
      "type": "Hero",
      "props": {
        "title": "Digital Products Built for Ambitious Brands",
        "backgroundColor": "#090d16",
        "backgroundGradient": true,
        "gradientFrom": "#090d16",
        "gradientTo": "#131b2e",
        "accentColor": "#6366f1",
        "textColor": "#ffffff",
        "borderRadius": 16,
        "animPreset": "fade-up",
        "animDuration": 700
      }
    },
    {
      "type": "ProductGrid",
      "props": {
        "sectionTitle": "Curated Seasonal Collection",
        "layout": "grid",
        "columns": "4",
        "cardBg": "#ffffff",
        "products": [ ... ]
      }
    }
  ],
  "root": {
    "props": {
      "title": "Home Page",
      "backgroundColor": "#ffffff",
      "fontFamily": "Inter"
```

```
    }
  }
}
```

3 How JSON Transforms Into Styled Visual Webpages

How does `backgroundColor: "#090d16"` become a visible dark background? This happens through our 3-stage engine:

Stage 1

Style Synthesizer

`buildBoxStyle()` turns raw props into CSS: `background`, `linear-gradient`, `borderRadius: "16px"`, and `backdropFilter`.

Stage 2

Token & Hover Engine

Resolves interactive micro-animations (like `hover-lift-md` or button shadows) via modular utility classes.

Stage 3

Animation Runtime

`AnimationWrapper` uses an `IntersectionObserver` to trigger GPU-accelerated entrance effects when scrolled into view.

4 Why This Architecture is Production-Grade

- **Tiny Database Footprint:** An entire multi-section, heavily styled page takes only **~20 KB of JSON** in NeonDB, instead of megabytes of duplicated HTML/CSS.
- **Infinite Editability:** Because styling is stored as clean properties, you can reopen any template months later and adjust any color, spacing, or text with zero friction.
- **Responsive By Default:** The components use modern fluid sizing (like CSS `clamp(...)` and responsive auto-fit grids), so one saved JSON document displays on Mobile, Tablet, and Desktop automatically.
- **Instant Client Cloning:** When someone clicks "Use Template", the backend simply duplicates the `editorData` JSON in under 50ms, giving the user their own editable website immediately.